

Claims + Sessions — Joint Configuration

One setup for the app: **claims** carry identity and drive authorization (admins create / edit / delete quotes, quote lines, and users; normal users only view), while **session** holds per-user working data. Both are configured together, below.

0. How the two fit together

This app uses claims and session as **one system with two responsibilities** — not two options. Every request carries both, and each has a distinct job:

Claims — `HttpContext.User`
WHO you are + your role.
In the encrypted auth cookie.
Drives `[Authorize]` / `User.IsInRole`.

Session — `HttpContext.Session`
WHAT you're working on.
Server-side (id in a cookie).
Read/written by your own code.

Kind of data	Configured as	Example in this app
Identity — who you are, your role	Claims (auth cookie)	UserId, Username, Admin/User — set once at login.
Temporary per-user working data	Session	A half-finished quote before you hit Save (Step 5).
Permanent data	Database	The saved quotes and quote lines.

The exact access rule this config enforces

Endpoint	Who may use it	Meaning
<code>/Login</code> (GET, POST, Logout)	Anyone	Must stay open — you can't log in if login needs a login.
<code>/Quote</code> GET, <code>/QuoteLine</code> GET	Any logged-in user	Viewing is allowed for everyone signed in.
<code>/Quote</code> POST / PUT / DELETE	Admin only	Create, edit, delete a quote.
<code>/QuoteLine</code> POST / PUT / DELETE	Admin only	Create, edit, delete a quote line.
<code>/User</code> GET + POST	Admin only	Only admins add users.

One interpretation to confirm

You said “edit and delete” but also “normal users can only view” — so I've made *creating* quotes and quote lines admin-only too. If you want normal users to *create* quotes, just leave the `[Authorize(Roles="Admin")]` off that one `POST`.

The law: the server enforces, the view only decorates

Hiding a Delete button is a courtesy — anyone can open dev-tools and run `fetch("/Quote/42", { method: "DELETE" })`. The only real lock is the `[Authorize]` check that runs on the server before your action. The view changes in Step 4 just stop showing controls a user can't use.

The steps below are one continuous build: Step 1 configures both mechanisms, Steps 2–4 wire up authorization from the claims, and Step 5 uses the session for working data. Do all of them.

Step 1 — Configure claims auth and session together (Program.cs)

Both are registered here, in the same file. Cookie authentication fills `HttpContext.User` so `[Authorize]` works; session provides the per-user working store. Your project already has the session lines — you're adding the authentication lines next to them.

`Program.cs` (services section)

```

using Microsoft.AspNetCore.Authentication.Cookies;

builder.Services.AddControllersWithViews();

// --- SESSION (working data) – already in your project ---
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.Cookie.Name = ".QuotesProject.Session";
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.IdleTimeout = TimeSpan.FromMinutes(30);
});

// --- CLAIMS AUTH (identity) – add this ---
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Login"; // unauthenticated page request → here
        options.AccessDeniedPath = "/Login"; // logged-in-but-forbidden → here
        options.ExpireTimeSpan = TimeSpan.FromMinutes(30);
        options.SlidingExpiration = true;

        // Your Save/Delete buttons use fetch – give them a real 401/403
        // instead of a 302 redirect to the login HTML.
        options.Events = new CookieAuthenticationEvents
        {
            OnRedirectToLogin = ctx =>
            {
                if (!ctx.Request.Headers.Accept.ToString().Contains("text/html"))
                    ctx.Response.StatusCode = 401;
                else
                    ctx.Response.Redirect(ctx.RedirectUri);
                return Task.CompletedTask;
            },
            OnRedirectToAccessDenied = ctx =>
            {
                if (!ctx.Request.Headers.Accept.ToString().Contains("text/html"))
                    ctx.Response.StatusCode = 403;
                else
                    ctx.Response.Redirect(ctx.RedirectUri);
                return Task.CompletedTask;
            }
        };
    });

```

Program.cs (pipeline section – this order is required)

```
app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthentication(); // builds HttpContext.User from the auth cookie
app.UseSession();        // makes HttpContext.Session available
app.UseAuthorization();   // must come AFTER UseAuthentication
```

Why this order: `UseAuthentication` works out *who* you are and must run before `UseAuthorization` decides *whether* you may proceed. `UseSession` only needs to run before the controllers do — sitting here between them is fine. All three run before `MapControllers`.

Step 2 — Issue claims at login (LoginController)

Login records identity as claims in the auth cookie. This is the one place your `int IsAdmin` becomes the role string that `[Authorize]` reads.

Controllers/LoginController.cs

```

using System.Security.Claims;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Login(LoginRequest1 login)    // now async
{
    if (!ModelState.IsValid)
        return BadRequest("Credentials are invalid");

    try
    {
        var user = _userService.ValidateUser(login.Username, login.Password);

        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
            new Claim(ClaimTypes.Name, user.Username),
            new Claim(ClaimTypes.Role, user.IsAdmin == 1 ? "Admin" : "User")    // 1/0 → role
        };

        var identity = new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme);
        var principal = new ClaimsPrincipal(identity);

        await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
principal);

        return Ok("User logged in successfully");
    }
    catch (ArgumentException ex)          { return BadRequest(ex.Message); }
    catch (UnauthorizedAccessException ex) { return Unauthorized(ex.Message); }
    catch (Exception ex)                  { return StatusCode(500, "Error logging in: " +
ex.Message); }
}

[HttpPost("Logout")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);    // drop
identity
    HttpContext.Session.Clear();    // drop this user's working data too
    return Ok("Logged out successfully");
}

```

What changed: the action is `async Task<IActionResult>` (you `await` sign-in); identity now travels in the auth cookie, so the old `Session.SetInt32/SetString` identity lines and the `string sessionId = ...` debug line are removed. Logout signs out (identity) and clears the session (working data). `ValidateUser`, password hashing, and antiforgery are untouched.

Step 3 — Protect the controllers with [Authorize]

Controllers/QuoteController.cs

```
using Microsoft.AspNetCore.Authorization;    // ← add

[ApiController]
[Route("[controller]")]
[Authorize]                                // every action needs a login (viewing included)
public class QuoteController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        // ← DELETE the old inline check:
        // if (HttpContext.Session.GetInt32("UserId") == null) return Redirect("/Login");
        var quotes = _quoteService.GetQuotes();
        return View(new QuoteViewModel { Quotes = quotes });
    }

    [HttpPost] [Authorize(Roles = "Admin")] public IActionResult CreateQuote(...) ...
    [HttpPut]  [Authorize(Roles = "Admin")] public IActionResult UpdateQuote(...) ...
    [HttpDelete("{quoteId}")] [Authorize(Roles = "Admin")] public IActionResult DeleteQuote(...)
    ...
}
```

Controllers/QuoteLineController.cs / UserController.cs

```
using Microsoft.AspNetCore.Authorization;

// QuoteLineController – same as QuoteController:
[Authorize]                                // class: view (GET) = any logged-in user
// [Authorize(Roles = "Admin")] on Create/Update/Delete line actions

// UserController – whole feature is admin-only:
[Authorize(Roles = "Admin")]               // class: both the GET page and the POST create
```

"Admin" must match the role-claim string from Step 2. Leave `LoginController` with no `[Authorize]` — it must stay reachable. Class-level `[Authorize(Roles="Admin")]` on `UserController` turns a normal user away from the page itself, not just the submit.

Step 4 — Views read the signed-in user

Views/Quote/Index.cshtml (same idea in QuoteLine/Index.cshtml)

```
@{
    ViewData["Title"] = "Quotes";
    var isAdmin = User.IsInRole("Admin");    // identity comes from the claims
}
```

Using `isAdmin` :

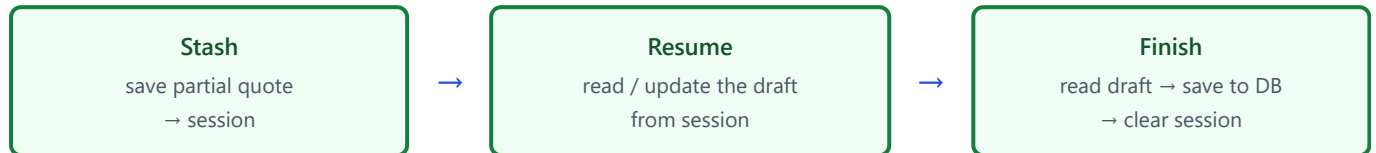
- Wrap the **Create Quote card** and the **Create User button** in `@if (isAdmin) { ... }`.
- Add `disabled="@(!isAdmin)"` to each editable input (customer, address, externalOrderNumber, orderDate, dueDate, invoiceDate; item/quantity/price/discount on QuoteLine).
- Wrap **Save** and **Delete** in `@if (isAdmin) { ... }`; keep **View** for everyone. Keep the Actions `<td>` present for all (the Quote table's DataTables config targets column index 8).

Views/Shared/_Layout.cshtml

```
@if (User.Identity.IsAuthenticated)
{
    <span>@User.Identity.Name @(User.IsInRole("Admin") ? "(Admin)" : "(User)")</span>
    <button class="btn btn-outline-secondary btn-sm" onclick="logout()">Logout</button>
    <script>
        function logout() {
            fetch("/Login/Logout", { method: "POST" }).then(() => window.location.href =
"/Login");
        }
    </script>
}
```

Step 5 — Store per-user working data in the session

Authorization is done; now the other half of the configuration does its job. Claims are frozen at login and shipped on every request — wrong for data that changes as the user works. A quote being built across several steps is *mutable working data*, so it lives in the session until the final Save. Both are present on the same request: `HttpContext.User` (claims) says who the user is; `HttpContext.Session` holds their draft.



Session stores `string`, `int`, or `byte[]` — not objects — so a whole `Quote` is serialised to JSON going in and back coming out. That `JsonSerializer` round-trip is the one genuinely-needed helper, shown inline to keep the flow visible.

```
Controllers/QuoteController.cs (add these actions next to the existing ones)
```



```

using System.Text.Json;

private const string DraftKey = "QuoteDraft"; // one session key, named once

// STASH – save what's been entered so far
[HttpPost("draft")]
[Authorize] // any logged-in user can build a draft
public IActionResult SaveDraft(Quote draft)
{
    HttpContext.Session.SetString(DraftKey, JsonSerializer.Serialize(draft));
    return Ok("Draft saved");
}

// RESUME – read the draft back
[HttpGet("draft")]
[Authorize]
public IActionResult GetDraft()
{
    var json = HttpContext.Session.GetString(DraftKey);
    if (json == null)
        return NoContent();

    return Ok(JsonSerializer.Deserialize<Quote>(json));
}

// FINISH – persist to the database, then clear the working data
[HttpPost("finish")]
[Authorize(Roles = "Admin")] // saving a REAL quote = admin (your rule)
public IActionResult FinishDraft()
{
    var json = HttpContext.Session.GetString(DraftKey);
    if (json == null)
        return BadRequest("No draft in progress.");

    _quoteService.CreateQuote(JsonSerializer.Deserialize<Quote>(json)); // → the database
    HttpContext.Session.Remove(DraftKey); // working data done
    return Ok("Quote saved");
}

```

- `SetString` / `GetString` are the built-in session accessors; `JsonSerializer` turns the `Quote` object into a string session can hold, and back.
- `Session.Remove(key)` clears one item; `Session.Clear()` (at logout) clears everything for that user.
- **Claims and session never collide:** `[Authorize]` uses the claims to decide who's allowed in; the session holds the draft for whoever that is. Each user has their own auth cookie *and* their own session, so drafts never mix between users.
- **The database is only touched at Finish** — everything before it is throwaway working state, which is exactly why it belongs in session and not the DB.

Rules of thumb for session data

Guideline	Why
Only <i>working</i> / <i>temporary</i> data	Identity is in claims; permanent data is in the database. Session is the scratchpad in between.
Keep it modest in size	The store is server memory; a big draft per user adds up. Large or long-lived drafts → a real draft row in the DB instead.
Clear when done (<code>Remove</code>) / at logout (<code>Clear</code>)	Stale drafts waste memory and can confuse the next flow.
Never assume it's durable	The in-memory store is wiped on app restart and expires after <code>IdleTimeout</code> (30 min). Fine for a draft.

How this configuration scales

Session lives in this app's memory today (`AddDistributedMemoryCache`). Growing to multiple servers means swapping *only* that one line for a shared store (`AddStackExchangeRedisCache(...)`) — no code above changes. Claims scale by sharing Data-Protection keys. The “claims for identity + session for working data” split is exactly the shape that grows cleanly.

6. File-by-file summary

File	What this config puts there
<code>Program.cs</code>	Session registration (yours) <i>and</i> <code>AddAuthentication().AddCookie(...)</code> ; pipeline order <code>UseAuthentication</code> → <code>UseSession</code> → <code>UseAuthorization</code> .
<code>LoginController.cs</code>	Login builds claims + <code>SignInAsync</code> ; logout does <code>SignOutAsync</code> + <code>Session.Clear()</code> . (Remove the old session identity writes + <code>sessionId</code> debug line.)
<code>QuoteController.cs</code>	<code>[Authorize]</code> (class) + <code>[Authorize(Roles="Admin")]</code> (writes); remove the inline session check in <code>Index()</code> ; add the draft actions (Step 5).
<code>QuoteLineController.cs</code>	<code>[Authorize]</code> + <code>[Authorize(Roles="Admin")]</code> on the write actions.
<code>UserController.cs</code>	<code>[Authorize(Roles="Admin")]</code> on the class.
Quote / QuoteLine / Layout views	<code>User.IsInRole("Admin")</code> for gating; <code>User.Identity</code> for the logged-in name.

7. Test it

Do this	You should see
1. Anonymous: open <code>/Quote</code> and <code>/User</code>	Both redirect to <code>/Login</code> .
2. Anonymous: <code>fetch("/Quote/1", {method: "DELETE"}).then(r=>r.status)</code>	401 (from the events in Step 1).
3. Log in as <code>user</code> / 1234, open <code>/Quote</code>	Read-only: no Create card, no Create-User button, no Save/Delete, inputs greyed, View works.
4. As <code>user</code> : <code>fetch("/Quote/1", {method: "DELETE"}).then(r=>r.status)</code>	403 , quote survives.
5. As <code>user</code> : type <code>/User</code> in the address bar	Blocked (403 / redirect).
6. Log in as <code>admin</code> / 1234	Create / edit / delete quotes & lines work; Create User works.
7. POST a draft to <code>/Quote/draft</code> , then GET <code>/Quote/draft</code>	Same draft back — it persisted in your session across requests.
8. Log out, then GET <code>/Quote/draft</code>	<code>NoContent</code> — logout's <code>Session.Clear()</code> discarded the draft.

The whole configuration, in one line

Identity is configured as **claims** (set at login, read by `[Authorize]` / `User.IsInRole`); working data is configured as **session** (mutable, server-side, cleared when done); permanent records go to the database. All three are set up together above and run side by side on every request. **This split is the standard shape of an ASP.NET Core app — the same three files (`Program.cs` config, a login that issues claims, `[Authorize]` on controllers) plus session for work-in-progress carry over to any app you build.**